

RAPPORT DE PROJET



PLATFORME DE CROWDFUNDING "TAMWILL"

Symfony

Doctrine

Twig

Tailwind CSS

Stripe

RÉALISÉ PAR :

- NIZAR TAOUSSI
- OTHMANE BAZ
- ANOUAR ELACHGAR

ENCADRÉ PAR :

Mme. Asmae Kassid

Table des matières

1. Introduction	5
1.1 Contexte.....	5
1.2 Objectifs.....	5
2. Analyse des Besoins	5
2.1 Acteurs du Système.....	5
2.2 Fonctionnalités Principales	5
3. Architecture Technique.....	6
3.1 Choix Technologiques	6
3.2 Structure de la Base de Données	6
4. STRUCTURE DES CONTRÔLEURS	8
4.1 IndexController.....	9
4.2 ProjectController	9
4.3 PaymentController	10
4.4 AdminController	10
4.5 SecurityController	11
4.6 ProfileController.....	11
4.7 ProjectDashboardController	11
5. Diagrammes UML	12
5.1 Diagramme de Classes.....	12
5.2 Diagramme de Cas d'Utilisation	13
6.1 Routes Publiques :	14
6.2 Routes Authentifiées:	14
7. SYSTÈME DE SÉCURITÉ	14
7.1 Authentification et Autorisation :	14
7.2 Protection CSRF:	15
7.3 Sécurité des Données:	15
7.3 Sécurité des Paiements :	15
8. SYSTÈME DE PAIEMENT	15
9. Description des Interfaces Utilisateur (Captures et Analyses).....	17
9.1 Page d'Accueil.....	17
9.2 Page de Détail d'un Projet	18
9.3 Modal de Donation.....	19

9.4 Page de Paiement (Checkout)	19
9.5 Dashboard Créateur.....	20
10. Gestion des Paiements et Flux Financiers.....	21
11. Difficultés Rencontrées et Solutions	22
11.1 Gestion de l'Anonymat dans le Classement.....	22
11.2 Intégration Stripe en Local.....	23
12.ÉVOLUTIONS FUTURES.....	24
13. Conclusion	24

Table des figures

Figure 1 : Structure de la table users – Base de données	6
Figure 2 : Structure de la table project – Base de données	7
Figure 3 : Structure de la table Contribution – Base de données	7
Figure 4 : Structure de la table payment – Base de données	7
Figure 5 : Structure de la table scoreboard– Base de données	8
Figure 6: Listes des contrôleurs	8
Figure 7 : Diagramme de classe	12
Figure 8: Diagramme de Cas d'Utilisation	13
Figure 9 : Capture de code : Création d'un PaymentIntent avec Stripe (PHP)	16
Figure 10 : Page d'Accueil	17
Figure 11 : Page de Détail d'un Projet	18
Figure 12 : Modal de Donation	19
Figure 13 : Page de Paiement (Checkout)	19
Figure 14 : Dashboard Créateur	20
Figure 15 : Capture de code : Gestion et agrégation des contributions utilisateurs....	22
Figure 16 : Dashboard Stripe	23

1. Introduction

1.1 Contexte

Le financement participatif est devenu un mécanisme incontournable de l'économie numérique, permettant la désintermédiation financière pour les porteurs de projets. Dans le cadre de ce projet académique, nous avons conçu "TamWill", une application web permettant de connecter des créateurs innovants avec une communauté de donateurs engagés.

1.2 Objectifs

L'objectif principal était de développer une plateforme robuste, sécurisée et esthétiquement moderne ("Neo-Brutalism" en noir et blanc) capable de gérer le cycle de vie complet d'une campagne de financement : de la création du projet à la collecte des fonds, jusqu'au virement final vers le créateur.

2. Analyse des Besoins

2.1 Acteurs du Système

Visiteur : Peut consulter les projets, voir la progression et s'inscrire.

Contributeur (Utilisateur) : Peut effectuer des dons via Stripe, choisir de rester anonyme, et commenter les projets.

Créateur de Projet : Peut lancer des campagnes, suivre les statistiques via un dashboard dédié et demander le retrait des fonds collectés.

Administrateur : Possède les droits globaux pour modérer le contenu et valider les virements bancaires.

2.2 Fonctionnalités Principales

Catalogue de Projets : Affichage dynamique (Masonry layout).

Système de Dons : Paiement sécurisé avec gestion de l'anonymat.

Gamification : Classement des meilleurs contributeurs ("Leaderboard") avec distinction des rangs.

Suivi Financier : Tableaux de bord avec graphiques d'évolution des dons.

3. Architecture Technique

3.1 Choix Technologiques

- **Framework Backend** : Symfony 6/7 pour sa robustesse et son architecture MVC stricte.
- **ORM** : Doctrine pour l'abstraction de la base de données et la gestion des entités.
- **Frontend** : Twig pour le templating, couplé à Tailwind CSS pour un design responsive et maintenable.
- **Base de Données** : MySQL.

3.2 Structure de la Base de Données

Le modèle de données s'articule autour des entités suivantes :

- User : Gère les informations de connexion et les rôles.

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra
1	id 🔑	int(11)			Non	Aucun(e)		AUTO_INCREMENT
2	name	varchar(255)	utf8mb4_unicode_ci		Non	Aucun(e)		
3	email 📧	varchar(180)	utf8mb4_unicode_ci		Non	Aucun(e)		
4	password	varchar(255)	utf8mb4_unicode_ci		Non	Aucun(e)		
5	role	varchar(50)	utf8mb4_unicode_ci		Non	Aucun(e)		
6	is_active	tinyint(1)			Non	Aucun(e)		
7	created_at	datetime			Non	Aucun(e)	(DC2Type:datetime_immutable)	
8	profile_picture	varchar(255)	utf8mb4_unicode_ci		Oui	NULL		
9	reset_token	varchar(255)	utf8mb4_unicode_ci		Oui	NULL		
10	reset_token_expires_at	datetime			Oui	NULL	(DC2Type:datetime_immutable)	

Figure 1 : Structure de la table users – Base de données

- **Project** : Contient les détails de la campagne (goal_amount, collected_amount, status, payout_status).

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra
1	id 🔑	int(11)			Non	Aucun(e)		AUTO_INCREMENT
2	creator_id 🔑	int(11)			Non	Aucun(e)		
3	title	varchar(255)	utf8mb4_unicode_ci		Non	Aucun(e)		
4	description	longtext	utf8mb4_unicode_ci		Non	Aucun(e)		
5	goal_amount	decimal(10,2)			Non	Aucun(e)		
6	collected_amount	decimal(10,2)			Oui	NULL		
7	start_date	datetime			Non	Aucun(e)		
8	end_date	datetime			Non	Aucun(e)		
9	status	varchar(50)	utf8mb4_unicode_ci		Non	Aucun(e)		
10	created_at	datetime			Non	Aucun(e)	(DC2Type:datetime_immutable)	
11	image	varchar(255)	utf8mb4_unicode_ci		Oui	NULL		
12	payout_status	varchar(50)	utf8mb4_unicode_ci		Oui	NULL		
13	rib	varchar(255)	utf8mb4_unicode_ci		Oui	NULL		
14	payout_requested_at	datetime			Oui	NULL		
15	payout_completed_at	datetime			Oui	NULL		

Figure 2 : Structure de la table project – Base de données

- **Contribution** : Table de liaison entre User et Project, stockant le montant et le flag is_anonymous.

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra
1	id 🔑	int(11)			Non	Aucun(e)		AUTO_INCREMENT
2	user_id 🔑	int(11)			Non	Aucun(e)		
3	project_id 🔑	int(11)			Non	Aucun(e)		
4	amount	decimal(10,2)			Non	Aucun(e)		
5	is_anonymous	tinyint(1)			Non	Aucun(e)		
6	created_at	datetime			Non	Aucun(e)	(DC2Type:datetime_immutable)	
7	payment_status	varchar(50)	utf8mb4_unicode_ci		Non	Aucun(e)		

Figure 3 : Structure de la table Contribution – Base de données

- **Payment** : Enregistre les détails techniques de la transaction (provider, transaction_id).

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra
1	id 🔑	int(11)			Non	Aucun(e)		AUTO_INCREMENT
2	contribution_id 🔑	int(11)			Non	Aucun(e)		
3	provider	varchar(255)	utf8mb4_unicode_ci		Non	Aucun(e)		
4	transaction_id	varchar(255)	utf8mb4_unicode_ci		Non	Aucun(e)		
5	amount	decimal(10,2)			Non	Aucun(e)		
6	status	varchar(50)	utf8mb4_unicode_ci		Non	Aucun(e)		
7	created_at	datetime			Non	Aucun(e)	(DC2Type:datetime_immutable)	

Figure 4 : Structure de la table payment – Base de données

- Scoreboard : Gère le classement des projets.

#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra
1	id 	int(11)			Non	Aucun(e)		AUTO_INCREMENT
2	project_id 	int(11)			Non	Aucun(e)		
3	rank_position	int(11)			Non	Aucun(e)		
4	total_amount	decimal(10,2)			Non	Aucun(e)		

Figure 5 : Structure de la table scoreboard- Base de données

Relations Entre Entités

User → Project : OneToMany (Un utilisateur peut créer plusieurs projets)

User → Contribution : OneToMany (Un utilisateur peut faire plusieurs contributions)

Project → Contribution : OneToMany (Un projet peut recevoir plusieurs contributions)

Project → Comment : OneToMany (Un projet peut avoir plusieurs commentaires)

Contribution → Payment : OneToOne (Une contribution correspond à un paiement)

4. STRUCTURE DES CONTRÔLEURS

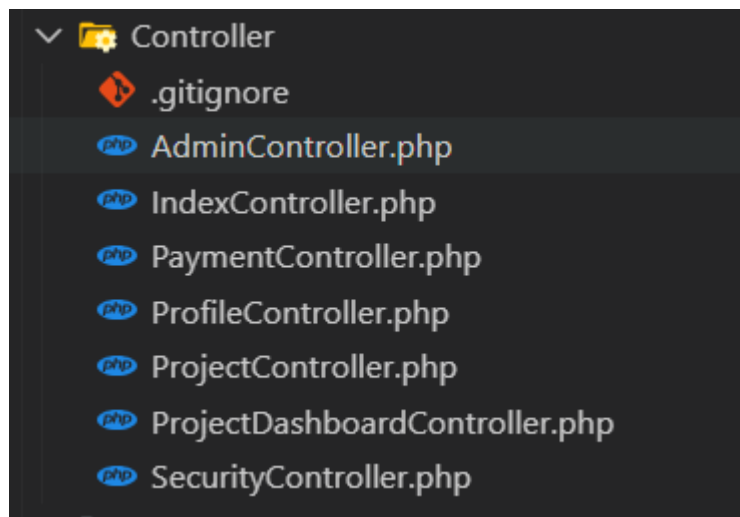


Figure 6: Listes des contrôleurs

4.1 IndexController

Route principale : / Nom : TamWill

Fonctionnalités :

- Affichage de la page d'accueil
- Récupération des 6 derniers projets
- Calcul des statistiques globales (montant total collecté, nombre de projets)

4.2 ProjectController

Routes principales :

- GET /campaigns - Liste des projets avec recherche et pagination
- POST /campaigns/{id}/comment - Ajout de commentaires
- GET/POST /project/create - Création de projet
- GET /project/{id} - Affichage détaillé d'un projet

Fonctionnalités clés :

- Gestion des projets : Création, affichage, recherche avec pagination (6 projets par page)
- Upload d'images : Traitement sécurisé avec génération de noms uniques
- Système de commentaires : Ajout de commentaires liés aux projets
- Affichage des contributions : Tri des contributeurs par montant et anonymisation
- Élévation de privilèges : Promotion automatique
USER → **CREATOR** lors de la création d'un projet

4.3 PaymentController

Routes :

- POST /payment/checkout/{id} - Initialisation du paiement Stripe
- GET /payment/success/{contributionId} - Confirmation de paiement

Fonctionnalités :

- Intégration Stripe : Utilisation de Payment Intents pour sécuriser les transactions
- Gestion des contributions : Création et suivi des contributions (statuts : pending, paid, failed)
- Contributions anonymes : Option pour masquer l'identité du contributeur
- Validation : Montant minimum de 5\$ par contribution
- Mise à jour automatique : Calcul du montant collecté en temps réel

4.4 AdminController

Route : /admin

Accès : ROLE_ADMIN uniquement

Fonctionnalités principales :

- Dashboard avec métriques :
 - Nombre total de projets, utilisateurs, commentaires
 - Montant total des donations
 - Graphiques temporels (donations et projets par mois)
- Gestion des utilisateurs : Liste, statuts, rôles
- Modération des projets : Supervision des projets actifs
- Gestion des commentaires : Suppression avec protection CSRF
- Traitement des demandes de versement : Approbation/rejet des payouts
- Analytics avancées : Données agrégées et visualisations Chart.js

4.5 SecurityController

Routes d'authentification :

- GET/POST / - Connexion (app_login)
- POST /register - Inscription
- GET /logout - Déconnexion
- GET/POST /forgot-password - Récupération de mot de passe
- GET/POST /reset-password/{token} - Réinitialisation

Fonctionnalités :

- Système d'inscription : Validation email, hachage bcrypt
- Récupération de mot de passe : Tokens sécurisés avec expiration
- Validation des données : Contrôles côté serveur
- Envoi d'emails : Templates professionnels via Mailer

4.6 ProfileController

Route : /profile

Fonctionnalités :

- Gestion de profil : Modification des informations personnelles
- Upload d'avatar : Gestion sécurisée des images de profil
- Changement de mot de passe : Validation de l'ancien mot de passe

4.7 ProjectDashboardController

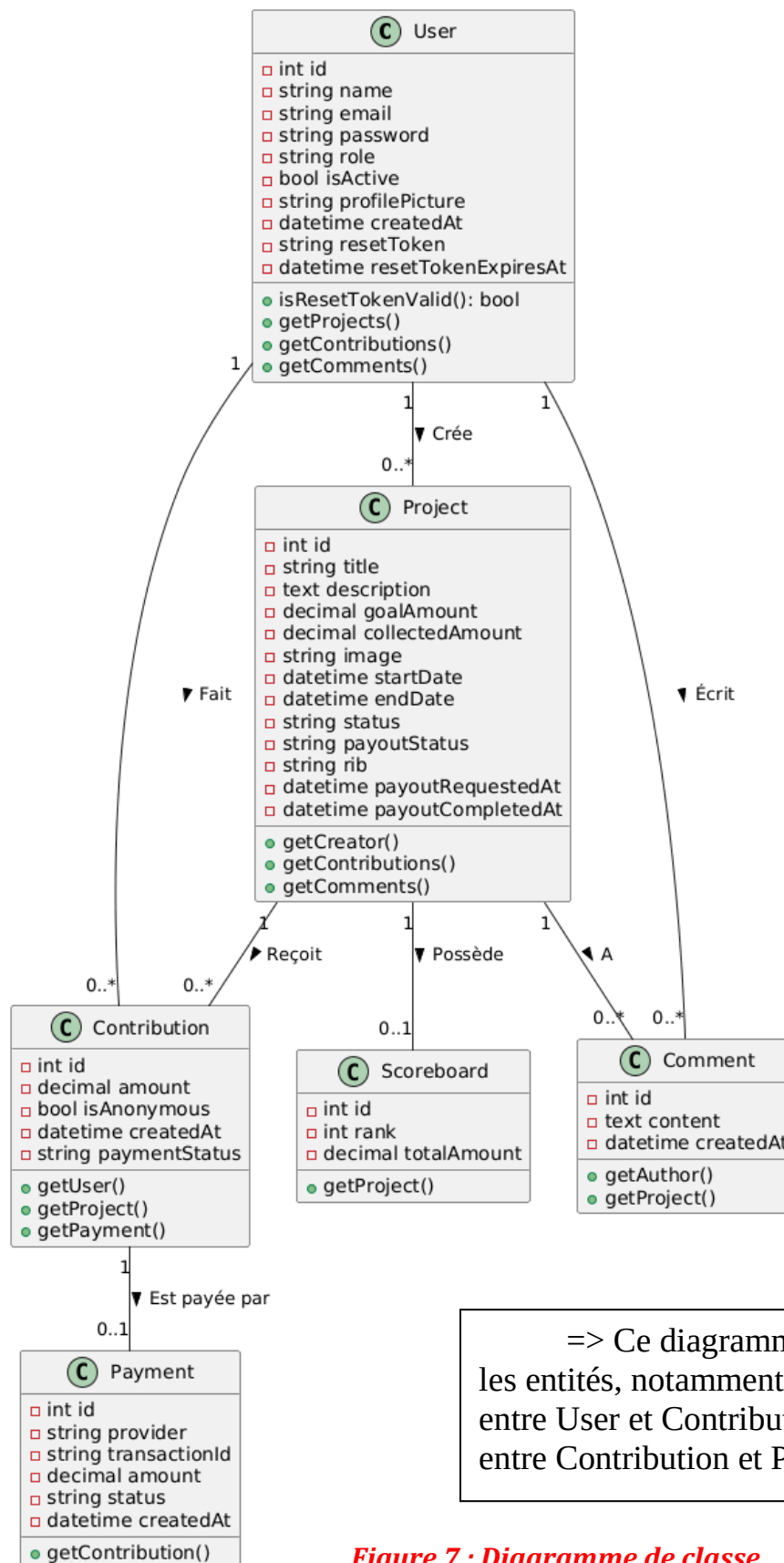
Route : /dashboard/project/{id}

Fonctionnalités :

- Tableau de bord projet : Vue détaillée pour les créateurs
- Suivi des contributions : Liste des contributeurs et montants
- Demandes de versement : Interface pour demander les fonds collectés
- Statistiques du projet : Progression vers l'objectif

5. Diagrammes UML

5.1 Diagramme de Classes



=> Ce diagramme illustre les relations entre les entités, notamment la relation One-to-Many entre User et Contribution, et la relation One-to-One entre Contribution et Payment.

Figure 7 : Diagramme de classe

5.2 Diagramme de Cas d'Utilisation

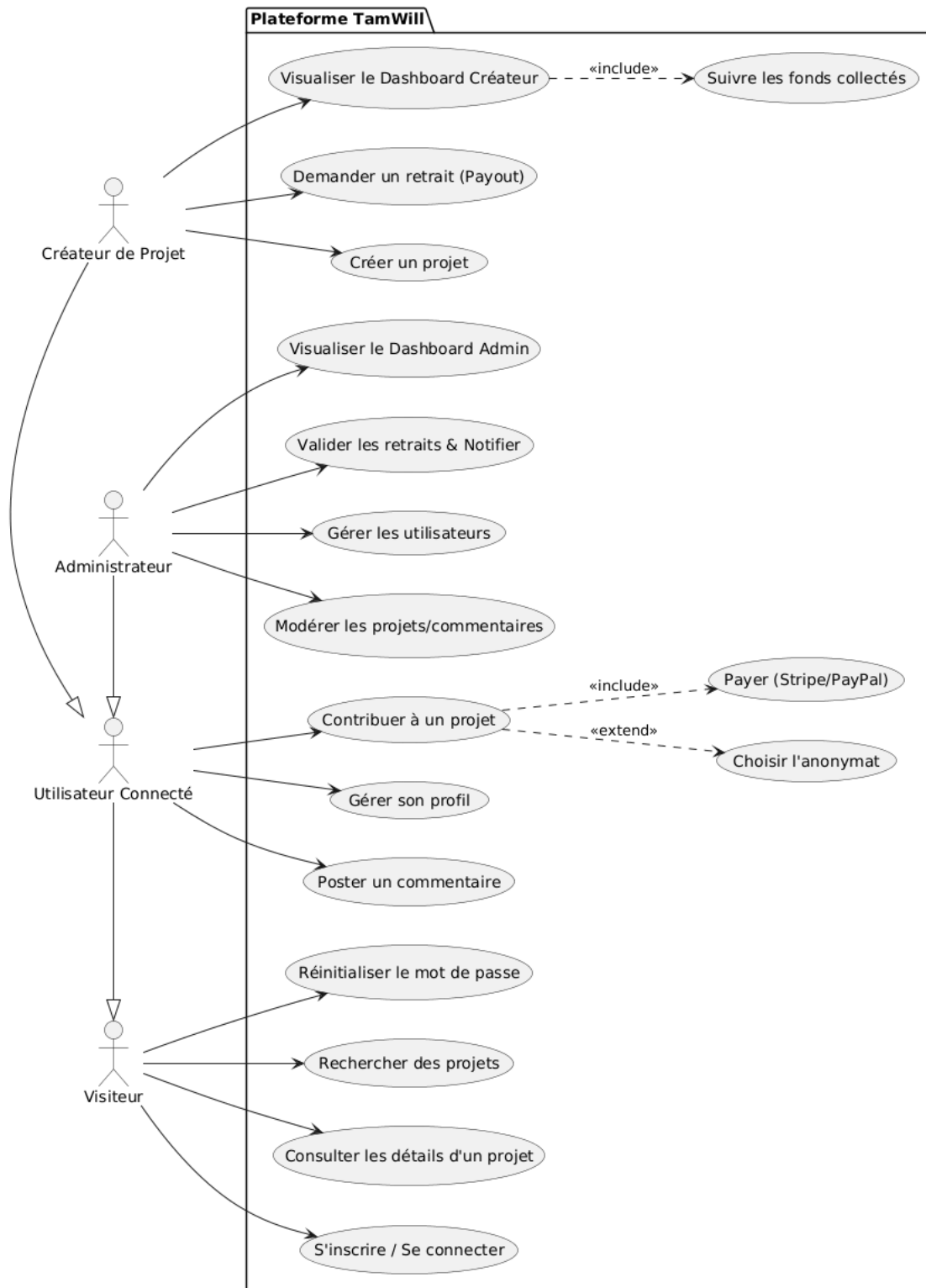


Figure 8: Diagramme de Cas d'Utilisation

=> Ce schéma détaille les interactions possibles pour chaque acteur, mettant en évidence la distinction entre un simple visiteur et un créateur disposant de fonctionnalités de gestion étendues.

6.1 Routes Publiques :

- | | |
|---------------------------|------------------------------|
| • / | Page d'accueil |
| • /campaigns | Liste des projets |
| • /campaigns?search=terme | Recherche de projets |
| • /project/{id} | Détail d'un projet |
| • /forgot-password | Récupération de mot de passe |
| • /reset-password/{token} | Réinitialisation |

6.2 Routes Authentifiées:

- | | |
|---------------------------|-----------------------|
| • /profile | Gestion de profil |
| • /project/create | Création de projet |
| • /campaigns/{id}/comment | Ajout de commentaires |
| • /payment/checkout/{id} | Processus de paiement |
| • /dashboard/project/{id} | Dashboard projet |

Routes Administrateur (ROLE_ADMIN)

- | | |
|---------------------------------------|-----------------------------|
| • /admin | Dashboard administrateur |
| • /admin/comment/{id}/delete | Suppression de commentaires |
| • /admin/users/{id}/toggle-status | Gestion statut utilisateurs |
| • /admin/projects/{id}/approve-payout | Approbation versements |

7. SYSTÈME DE SÉCURITÉ

7.1 Authentification et Autorisation :

- **UserInterface & PasswordAuthenticatedUserInterface** implémentés
- **Rôles hiérarchiques :**
 - **ROLE_USER** : Utilisateurs standard
 - **ROLE_CREATOR** : Créateurs de projets
 - **ROLE_ADMIN** : Administrateurs
- **Protection des routes** avec `#[IsGranted('ROLE_ADMIN')]`

7.2 Protection CSRF:

- Tokens CSRF sur tous les formulaires critiques
- Validation côté serveur pour les actions sensibles
- Protection des API endpoints (suppression, modification)

7.3 Sécurité des Données:

- **Hachage bcrypt** des mots de passe
- **Tokens de réinitialisation** avec expiration (1 heure)
- **Validation stricte** des entrées utilisateur
- **Sanitisation** des uploads de fichiers
- **Contrôles d'accès** granulaires

7.3 Sécurité des Paiements :

- **Stripe Payment Intents** (PCI DSS compliant)
- **Webhook validation** pour la confirmation des paiements
- **Métadonnées sécurisées** pour tracer les transactions

8. SYSTÈME DE PAIEMENT

Intégration Stripe :

La plateforme utilise Stripe Elements avec Payment Intents pour une sécurité maximale :

- **Processus de Paiement :**

1. Création de la contribution (statut: pending)
2. Génération du Payment Intent Stripe
3. Interface de paiement sécurisée côté client
4. Confirmation automatique via webhook
5. Mise à jour du statut (pending → paid)
6. Enregistrement du paiement avec transaction ID

- **Configuration Stripe**

```
\Stripe\Stripe::setApiKey($_ENV['STRIPE_SECRET_KEY']);

$paymentIntent = \Stripe\PaymentIntent::create([
    'amount' => $amount * 100,
    'currency' => 'usd',
    'automatic_payment_methods' => ['enabled' => true],
    'metadata' => [
        'contribution_id' => $contribution->getId(),
        'project_id' => $project->getId()
    ],
]);
```

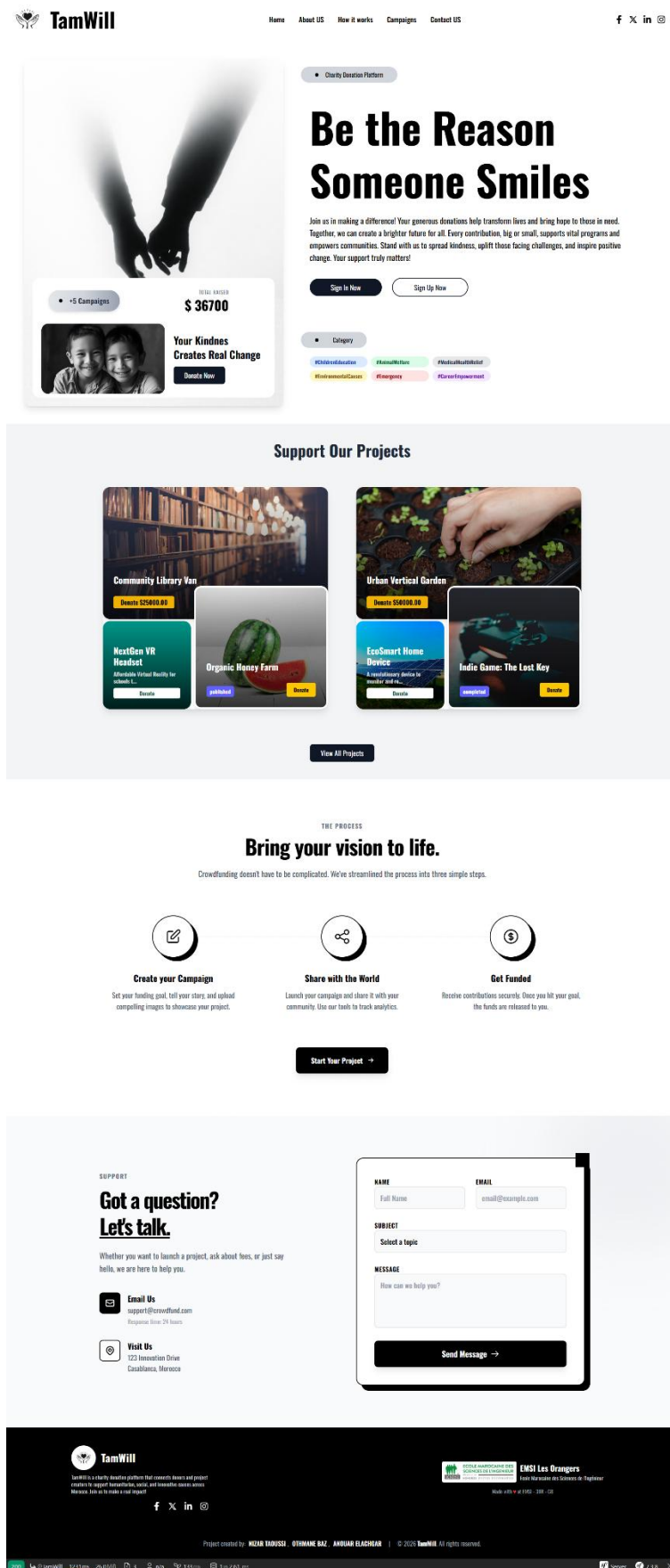
Figure 9 : Capture de code : Création d'un PaymentIntent avec Stripe (PHP)

- **Gestion des Contributions :**

- Montant minimum : 5 USD
- Contributions anonymes : Option pour masquer l'identité
- Statuts de paiement : pending, paid, failed
- Historique complet des transactions

9. Description des Interfaces Utilisateur (Captures et Analyses)

9.1 Page d'Accueil



Analyse de l'interface :
La page d'accueil de "TamWill" incarne l'identité visuelle "Néo-Brutaliste" du projet. Le choix d'une palette strictement monochrome (Noir & Blanc) permet de hiérarchiser l'information sans distraction : le regard est immédiatement guidé vers les éléments essentiels, à savoir les appels à l'action (Boutons "Sign In/Up") et les visuels des campagnes.

Figure 10 : Page d'Accueil

9.2 Page de Détail d'un Projet

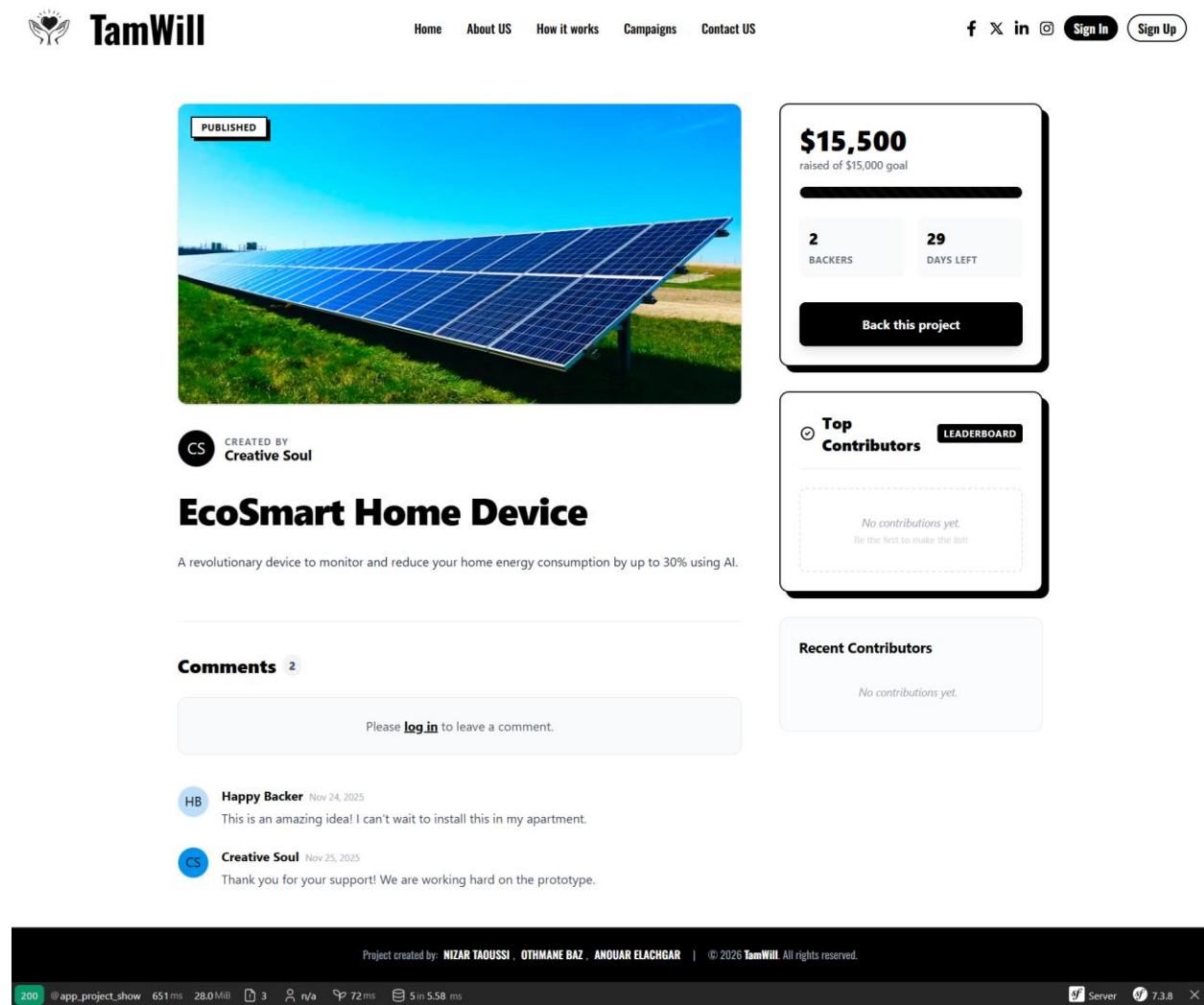


Figure 11 : Page de Détail d'un Projet

- Analyse de l'interface : Cette page est structurée en deux colonnes.
- Gauche : Contenu riche (Description, Image, Commentaires). Les commentaires sont clairement séparés pour une meilleure lisibilité.
- Droite (Sticky) : Une carte "flottante" qui reste visible lors du défilement. Elle contient les métriques clés (Montant collecté, Objectif, Nombre de contributeurs) et le bouton d'action principal "Back this project".
- Leaderboard : Une section "Top Contributors" utilise des icônes de trophées et un style distinct pour les 3 meilleurs donateurs, favorisant la compétition bienveillante.

9.3 Modal de Donation

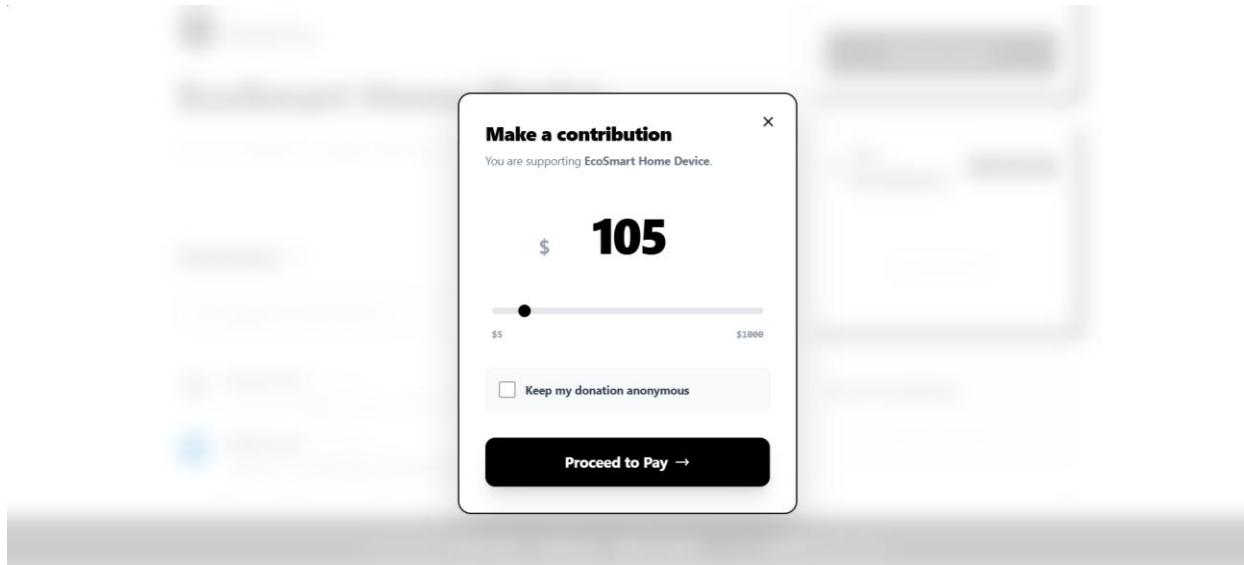


Figure 12 : Modal de Donation

Analyse de l'interface : Le processus de don débute par une modale interactive. Plutôt qu'un simple champ texte, un curseur (range slider) permet de sélectionner le montant de manière ludique, avec un retour visuel immédiat en gros caractères. Une case à cocher "Donate anonymously" est stratégiquement placée avant la validation pour rassurer l'utilisateur sur sa vie privée.

9.4 Page de Paiement (Checkout)

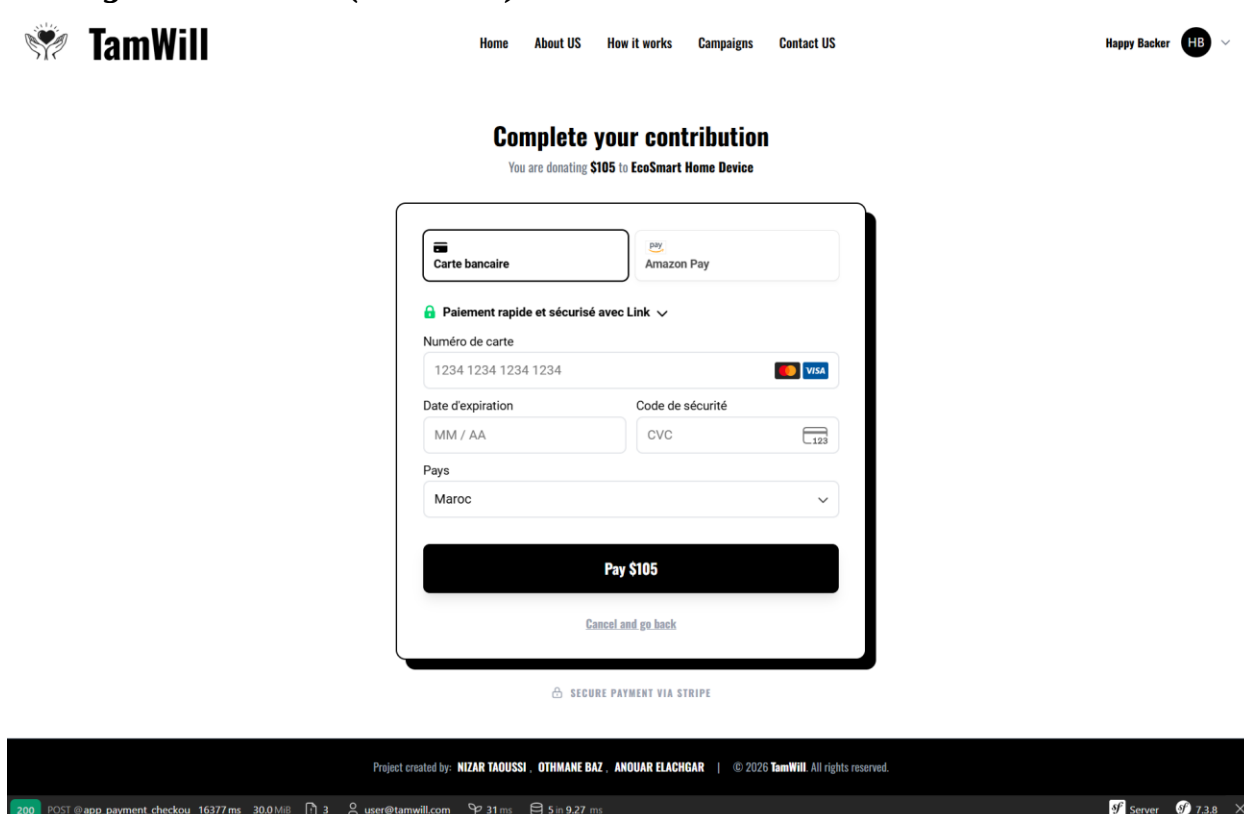


Figure 13 : Page de Paiement (Checkout)

Analyse de l'interface : L'intégration de Stripe Elements permet de conserver l'utilisateur sur le site sans redirection externe visible. Le formulaire reprend le style "Brutalist" du site (bordures noires épaisses, ombres dures). Un récapitulatif clair du montant et du projet bénéficiaire est affiché en tête de page pour éviter toute confusion.

9.5 Dashboard Créateur

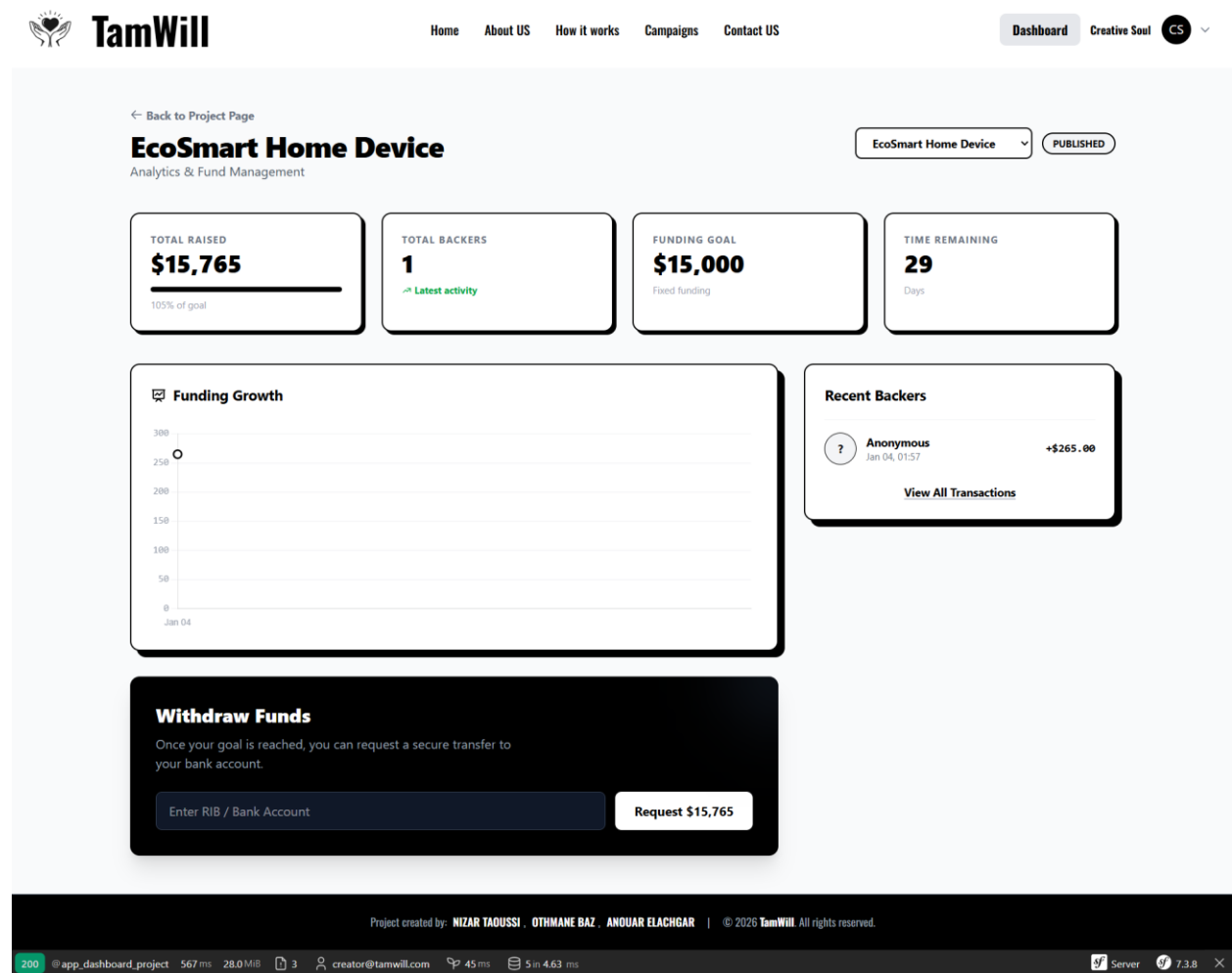


Figure 14 : Dashboard Créateur

Analyse de l'interface : Le tableau de bord offre une vue synthétique de la performance de la campagne.

Cartes KPI : Quatre cartes en haut résument les chiffres clés (Total levé, % de l'objectif, etc.).

Graphique : Un graphique linéaire (réalisé avec Chart.js) permet de visualiser la dynamique des dons dans le temps.

Zone de Retrait : Un bouton conditionnel permet de demander le virement des fonds (Request Payout) uniquement lorsque l'objectif est atteint, sécurisant ainsi le modèle économique.

10. Gestion des Paiements et Flux Financiers

L'application intègre une stratégie de paiement hybride et un modèle de reversement manuel (Escrow) :

- Collecte (Pay-in) : Les fonds sont collectés via Stripe et stockés sur le compte de la plateforme. Le statut du paiement est suivi en base de données (pending -> paid).
- Reversement (Payout) : Lorsqu'un projet réussit, le créateur soumet son RIB via le dashboard. L'administrateur valide la demande et effectue le virement manuellement, mettant à jour le statut du projet (payoutStatus: paid).

11. Difficultés Rencontrées et Solutions

11.1 Gestion de l'Anonymat dans le Classement

```
foreach ($contributions as $contribution) {
    $user = $contribution->getUser();

    $isAnonymous = method_exists($contribution, 'isIsAnonymous') ? $contribution->isIsAnonymous() : false;

    if ($isAnonymous || $user === null) {
        $userId = 'anon_' . $anonIndex++;

        $userContributions[$userId] = [
            'user' => null,
            'amount' => $contribution->getAmount(),
            'isAnonymous' => true,
            'createdAt' => $contribution->getCreatedAt(),
        ];
    } else {
        $userId = $user->getId();

        if (!isset($userContributions[$userId])) {
            $userContributions[$userId] = [
                'user' => $user,
                'amount' => 0,
                'isAnonymous' => false,
                'createdAt' => $contribution->getCreatedAt(),
            ];
        }

        $userContributions[$userId]['amount'] += $contribution->getAmount();
    }
}
```

Figure 15 : Capture de code : Gestion et agrégation des contributions utilisateurs

Problème : Comment afficher un classement juste sans révéler l'identité des donateurs anonymes ? Solution : Une logique spécifique a été implémentée dans le ProjectController. Les dons publics d'un même utilisateur sont agrégés (sommés), tandis que les dons anonymes sont traités comme des entités distinctes (anon_1, anon_2) pour empêcher l'identification par déduction via le montant total.

11.2 Intégration Stripe en Local

Environnement de test Vous effectuez des tests dans un environnement de test, où vous pouvez essayer les fonctionnalités Stripe. [Basculer sur le compte de production](#)

Nouvelle entreprise New business

Rechercher Les environnements de test sont le nouveau moyen d'effectuer des tests. [En savoir plus](#)

Accueil Soldes Transactions Clients Catalogue de produits

Raccourcis Profils

Produits Payments Billing Reporting Plus

Transactions

[Créer un paiement](#) [Analyser](#)

[Tous](#) 23 Réussis 8 Remboursés 0 Contestés 0 En échec 1 Non capturés 0

[Date et heure](#) [Montant](#) [Devise](#) [État](#) [Moyen de paiement](#) [Plus de filtres](#) [Exporter](#) [Modifier les colonnes](#)

	Montant	Moyen de paiement	Description	Client	Date	Date du remboursement	Mot
☐	265,00 \$US USD Réussi ✓	**** 4242	pi_3S1fd1EnEyabXy920Mou308W	taoussinzae7@gmail.com	4 janv. à 00:57	—	...
☐	105,00 \$US USD Incomplet ⚠	—	pi_3S1fZrEnEyabXy920uWfXJT	taoussinzae7@gmail.com	4 janv. à 00:53	—	...
☐	10 000,00 \$US USD Réussi ✓	**** 4242	pi_3S144HEnEyabXy9210sz8H65	taoussinzae7@gmail.com	2 janv. à 08:51	—	...
☐	1 000,00 \$US USD Réussi ✓	**** 4242	pi_3S12veEnEyabXy921mJUZIBH	taoussinzae7@gmail.com	2 janv. à 07:38	—	...
☐	1 000,00 \$US USD Incomplet ⚠	—	pi_3S12v8EnEyabXy921t41Mu1m	taoussinzae7@gmail.com	2 janv. à 07:37	—	...
☐	50,00 \$US USD Incomplet ⚠	—	pi_3Sk1twEnEyabXy921DLuJzzg	taoussinzae7@gmail.com	1 janv. à 13:26	—	...
☐	50,00 \$US USD Incomplet ⚠	—	pi_3Sk1r8EnEyabXy921tSo3zXz	taoussinzae7@gmail.com	1 janv. à 13:24	—	...

Affichage 1-20 de 23 éléments [Précédent](#) [Suivant](#)

dashboard.stripe.com/.../pi_3S1fd1EnEyabXy920Mou308W

Figure 16 : Dashboard Stripe

Problème : Stripe nécessite des URLs publiques (HTTPS) pour les webhooks et les images produits. Solution : Utilisation du mode "Test" de Stripe et simulation des processus de validation directement dans le contrôleur de retour (success_url), sans dépendre exclusivement des webhooks pour l'environnement de développement.

12.ÉVOLUTIONS FUTURES

Fonctionnalités Prévues

- **Système de notifications** en temps réel
- **API REST** pour applications mobiles
- **Système de récompenses** pour contributeurs
- **Module de reporting** avancé
- **Intégration réseaux sociaux**

Améliorations Techniques

- **Migration vers Symfony 8.0**
- **Implémentation de Redis** pour le cache

13. Conclusion

Le projet "TamWill" démontre la capacité de Symfony à gérer des logiques métiers complexes (paiements, gestion de fonds) tout en offrant une interface utilisateur fluide et distinctive. L'architecture modulaire mise en place permet d'envisager sereinement des évolutions futures, telles que l'automatisation des versements via Stripe Connect ou l'ajout d'un système de récompenses par paliers.

Au nom de mon groupe, je tiens à souligner que cette aventure a été autant technique qu'humaine. Du design audacieux "Néo-Brutaliste" à la logique complexe des dons anonymes, chaque ligne de code reflète notre engagement commun. Nous sommes fiers de livrer aujourd'hui une application aboutie, témoin de notre capacité à transformer une idée en un produit fonctionnel et innovant.

 ACCÈS AU CODE SOURCE

L'intégralité du travail réalisé par le groupe est disponible sur notre dépôt GitHub. Scannez ce code pour accéder aux sources :

